

DBMS Lab Assignment – 6

PL/SQL Functions

What is a Function in PL/SQL?

A function is a named PL/SQL Block which is similar to a procedure. The major difference between a procedure and a function is, a function must always return a value, but a procedure may or may not return a value.

Syntax:-

```
CREATE [OR REPLACE] FUNCTION function_name [(  
    parameter_1 [IN] [OUT] data_type,  
    parameter_2 [IN] [OUT] data_type,  
    parameter_N [IN] [OUT] data_type)  
    RETURN return_data_type IS  
    --the declaration statements  
BEGIN  
    -- the executable statements  
    return return_data_type;  
EXCEPTION  
    -- the exception-handling statements  
END;  
/
```

- *Function-name* specifies the name of the function.
- [or replace] option allows the modification of an existing function.
- The optional parameter list contains name, mode and types of the parameters. In represents the value that will be passed from outside and out represents the parameter that will be used to return a value outside of the procedure.
- The function must contain a **return** statement.
- The *return* clause specifies the data type you are going to return from the function.
- *Function-body* contains the executable part.
- The as keyword is used instead of the is keyword for creating a standalone function.

A **subprogram** is a program unit/module that performs a particular task. These subprograms are combined to form larger programs. This is basically called the 'Modular design'. A subprogram can be invoked by another subprogram or program which is called the calling program.

3. Write a PL/SQL function to define total number of books in SIU Library

```
mysql> DELIMITER //
mysql> CREATE FUNCTION GetTotalBooks()
  -> RETURNS INT
  -> READS SQL DATA
  -> BEGIN
  ->     DECLARE total_books INT;
  ->     SELECT COUNT(*) INTO total_books FROM books;
  ->     RETURN total_books;
  -> END //
Query OK, 0 rows affected (0.01 sec)

mysql> DELIMITER ;
mysql> SELECT GetTotalBooks();
+-----+
| GetTotalBooks() |
+-----+
|                22 |
+-----+
1 row in set (0.02 sec)
```

4. Write a PL/SQL function to find average of 4 numbers.

```
mysql> DELIMITER //
mysql> CREATE FUNCTION GetAverageOfFour(n1 FLOAT, n2 FLOAT, n3 FLOAT, n4 FLOAT)
  -> RETURNS FLOAT
  -> DETERMINISTIC
  -> BEGIN
  ->     RETURN (n1 + n2 + n3 + n4) / 4.0;
  -> END //
Query OK, 0 rows affected (0.00 sec)

mysql> DELIMITER ;
mysql> SELECT GetAverageOfFour(10, 20, 30, 40);
+-----+
| GetAverageOfFour(10, 20, 30, 40) |
+-----+
|                                25 |
+-----+
1 row in set (0.00 sec)
```

5. Write a PL/SQL function to find factorial.

```
mysql> DELIMITER //
mysql> CREATE FUNCTION GetFactorial(n INT)
  -> RETURNS INT
  -> DETERMINISTIC
  -> BEGIN
  ->     DECLARE fact INT DEFAULT 1;
  ->     DECLARE i INT DEFAULT 1;
  ->
  ->     WHILE i <= n DO
  ->         SET fact = fact * i;
  ->         SET i = i + 1;
  ->     END WHILE;
  ->     RETURN fact;
  -> END //
Query OK, 0 rows affected (0.01 sec)

mysql> DELIMITER ;
mysql> SELECT GetFactorial(5);
+-----+
| GetFactorial(5) |
+-----+
|              120 |
+-----+
1 row in set (0.00 sec)
```

6. Write a PL/SQL function to find the name of the library which has the cheapest books.

```
mysql> DELIMITER //
mysql> CREATE FUNCTION GetCheapestLibrary()
-> RETURNS VARCHAR(100)
-> READS SQL DATA
-> BEGIN
->     DECLARE cheapest_lib VARCHAR(100);
->
->     SELECT l.lname INTO cheapest_lib
->     FROM ilibrary l
->     JOIN books b ON l.Lid = b.Lid
->     ORDER BY b.Price ASC
->     LIMIT 1;
->
->     RETURN cheapest_lib;
-> END //
Query OK, 0 rows affected (0.01 sec)

mysql> DELIMITER ;
mysql> SELECT GetCheapestLibrary();
+-----+
| GetCheapestLibrary() |
+-----+
| SIT Library          |
+-----+
1 row in set (0.01 sec)
```

7. Write a PL/SQL function to find total no. of books issued by S-IT Staff.

```
mysql> DELIMITER //
mysql> CREATE FUNCTION GetBooksIssuedBySIT()
-> RETURNS INT
-> READS SQL DATA
-> BEGIN
->     DECLARE issue_count INT;
->
->     SELECT COUNT(*) INTO issue_count
->     FROM issue i
->     JOIN staff s ON i.memid = s.memid
->     JOIN department d ON s.deptid = d.Deptid
->     WHERE d.deptname = 'SIT';
->
->     RETURN issue_count;
-> END //
Query OK, 0 rows affected (0.00 sec)

mysql> DELIMITER ;
mysql> SELECT GetBooksIssuedBySIT();
+-----+
| GetBooksIssuedBySIT() |
+-----+
| 0 |
+-----+
1 row in set (0.01 sec)
```

Conclusion:

I learned how to create and execute user-defined functions in MySQL to automate mathematical operations and complex database queries. I gained practical experience setting delimiters, declaring variables, and using control structures like loops and conditional statements. Furthermore, I learned how to embed JOIN operations inside functions to retrieve specific data across multiple relational tables. Finally, troubleshooting errors taught me the importance of using the DESCRIBE command to verify exact table schemas before writing SQL logic.